META-GRAD: META-LEARNING A LOW-RANK GATING POLICY FOR FINE-GRAINED GRADIENT AGREEMENT

Anonymous authors
Paper under double-blind review

ABSTRACT

Multi-task learning (MTL) seeks to enhance generalization and efficiency by training a single model on multiple objectives. However, this process is frequently undermined by negative transfer, where conflicting task gradients result in destructive updates that degrade overall performance. Prevailing methods for mitigating this issue are polarized: reactive gradient surgery techniques like PCGrad and CAGrad operate at a coarse, vector-level, potentially discarding valuable parameter-specific information, whereas architectural methods introduce rigid structural changes that are computationally expensive and slow to adapt. This paper introduces Meta-Grad, a novel framework that bridges this divide by reformulating gradient aggregation as a meta-learned, parameter-level policy. We propose learning task-specific, low-rank matrices that generate fine-grained gating masks for each task's gradient. This policy is optimized proactively within a single training batch via an efficient meta-objective that maximizes worst-case task improvement, thereby seeking a gradient update that represents a genuine consensus direction. By learning to dynamically arbitrate gradient conflicts at the parameter level, Meta-Grad avoids the limitations of both coarse-grained projections and static architectural choices. We validate our approach through comprehensive experiments on standard MTL benchmarks, demonstrating that Meta-Grad consistently discovers superior Pareto-optimal fronts compared to state-of-the-art methods.

1 INTRODUCTION

Multi-task learning (MTL) has emerged as a powerful paradigm in machine learning, offering the potential to improve data efficiency, reduce overfitting, and enhance model generalization by leveraging shared representations across a diverse set of tasks **?**. The central idea is that knowledge acquired for one task can provide a useful inductive bias for others. Despite its promise, naively combining tasks within a single network often leads to a phenomenon known as negative transfer, where the performance on some or all tasks degrades compared to training them independently **?**. This is frequently caused by conflicting gradients, where the update direction beneficial for one task is detrimental to another.

To address this challenge, the community has largely pursued two distinct lines of research. The first involves gradient manipulation, where per-task gradients are altered at each training step to mitigate conflicts. Prominent methods like PCGrad **?** and CAGrad **?** operate by projecting gradients or finding an update within a specific cone of agreement. While effective, these gradient surgery techniques are inherently reactive and operate at the vector level. By treating the entire gradient vector for a shared layer as a single entity, they may discard valuable, fine-grained information where tasks agree on some parameters but conflict on others.

The second major approach involves architectural adaptation. This includes static strategies that predetermine which tasks should be grouped together **?** and dynamic methods that modify the network structure during training, such as by merging or forking task-specific layers **??**. These methods can be powerful but often incur significant computational overhead and adapt slowly, typically over multiple epochs, failing to respond to the rapidly changing dynamics of gradient conflicts within a single training batch.

We identify a critical gap between these two extremes: a need for a method that can dynamically arbitrate gradient conflicts at the parameter level, proactively, and within a single training step. While some work has recognized that parameters have task-specific importance **?**, it has often relied on rigid, heuristic-based schemes. In this paper, we propose Meta-Grad, a framework that recasts gradient aggregation as a meta-learned policy. Instead of directly manipulating final gradient vectors, we learn a policy that constructs the aggregated gradient from its constituent parts. This policy generates task-specific, fine-grained gating masks for each parameter, effectively learning to route gradient information. To ensure scalability, these masks are generated from low-rank matrices, analogous to applying LoRA adapters to the gradients themselves. Crucially, the policy parameters are optimized via an efficient, single-batch meta-objective designed to find a consensus update that benefits the most conflicting task.

Our contributions can be summarized as follows:

- **Novel Gating Policy Framework.** We propose Meta-Grad, a novel framework that learns a proactive, parameter-level gating policy to aggregate gradients in MTL, bridging the gap between coarse gradient surgery and slow architectural methods.
- **Low-Rank Parametrization.** We introduce a parameter-efficient, low-rank factorization for the gating policy, ensuring its scalability to large-scale models.
- **Efficient Meta-Objective.** We design an efficient, single-batch meta-objective, the Worst-Case Improvement Objective, which trains the gating policy to find consensus directions without the overhead of bi-level optimization.
- **State-of-the-Art Performance.** Through extensive experiments, we demonstrate that Meta-Grad achieves state-of-the-art performance, discovering superior Pareto-optimal fronts compared to a wide range of strong baselines.

2 Related Work

The challenge of mitigating negative transfer in multi-task learning has inspired a diverse range of solutions, which can be broadly categorized into gradient manipulation, architectural adaptation, and heuristic-based weighting schemes.

Gradient Manipulation Methods This line of work focuses on modifying the raw per-task gradients before they are used to update the shared model parameters. A seminal work in this area is PCGrad **?**, which identifies conflicting gradients (those with a negative cosine similarity) and projects the gradient of one task onto the normal plane of the other. This prevents the update for one task from directly interfering with another. Building on this, CAGrad **?** formulates the problem as finding a single update vector within the convex hull of task gradients that minimizes the loss for the most-conflicting task. While these methods have proven effective, they operate on the entire gradient vector for a set of shared parameters. Meta-Grad diverges from these vector-level approaches by learning a parameter-wise gating policy. This allows it to preserve useful updates for certain parameters even when the overall gradient vectors for a layer are in conflict, offering a more nuanced resolution mechanism.

Architectural Methods An alternative approach is to modify the network architecture itself to control information sharing and reduce task interference. Some methods rely on static task grouping, using task similarity metrics to decide which tasks should share a backbone **?**. More recent works have proposed dynamic architectures. For instance, MTSL **?** learns to group similar task layers during training, while ForkMerge **?** periodically forks the model into branches and merges them to filter detrimental updates. Other approaches, like Recon **?**, identify layers with high gradient conflict and convert them into task-specific modules. These methods provide 'hard' parameter specialization but are often computationally expensive and adapt over longer timescales (multiple epochs). Meta-Grad offers a 'soft' form of specialization by dynamically routing gradient information within a static architecture at every training step, providing greater flexibility with lower overhead.

Heuristic and Learned Weighting Schemes This category includes methods that adjust the contribution of each task's loss or gradient to the final update. This ranges from static grid searches **?** to dynamic schemes based on uncertainty **?** or loss dynamics. More related to our work is the concept of identifying task-specific parameter importance. The work of Jeong and Yoon **?** introduced the idea of 'task priority' to determine the contributions of parameters across tasks. However, their method relies on a heuristic-based, two-phase scheme based on connection strength. Meta-Grad advances this concept by replacing the heuristic with a fully learnable, end-to-end policy. The gating masks in Meta-Grad are not based on a fixed heuristic but are instead the output of a policy that is directly optimized via a meta-objective to resolve conflicts and find Pareto-optimal solutions.

3 BACKGROUND

To understand our proposed method, we first review the foundational concepts of multi-task learning and formally define the problem of gradient aggregation from a multi-objective optimization perspective.

Multi-task learning (MTL) is a machine learning paradigm where a single model is trained to perform multiple tasks simultaneously. A typical MTL network consists of a shared backbone or encoder, which learns a common feature representation, and several task-specific heads or decoders that map the shared representation to the desired output for each task. By sharing parameters across tasks, MTL can leverage commonalities and differences to improve learning efficiency and prediction accuracy **?**.

Problem Setting and Notation Let us consider an MTL setup with N tasks, denoted by $\mathcal{T} = \{T_1, T_2, \dots, T_N\}$. For each task T_i, we have a corresponding loss function L_i. The model parameters θ are partitioned into a set of shared parameters W_s and a set of task-specific parameters W_i for each task T_i, such that $\theta = \{W_s, W_1, \dots, W_N\}$. The primary optimization challenge lies in updating the shared parameters W_s, as they receive update signals from all N tasks.

At each training step, we compute the per-task gradient for the shared parameters, $\nabla_s L_i \equiv \nabla_{W_s} L_i(W_s, W_i)$. A naive approach is to simply average these gradients:

$$W_s \leftarrow W_s - \eta \cdot \frac{1}{N} \sum_{i=1}^{N} \nabla_s L_i$$

where η is the learning rate. This approach often fails due to negative transfer **?**, where gradients conflict (i.e., have a negative cosine similarity: $\langle \nabla_s L_i, \nabla_s L_j \rangle < 0$). An update that decreases L_i may increase L_j, leading to suboptimal performance.

Multi-Objective Optimization Perspective The MTL problem can be framed as a multi-objective optimization (MOO) problem, where the goal is to simultaneously minimize a vector of loss functions $[L_1(\theta), L_2(\theta), \dots, L_N(\theta)]$. In MOO, there is typically no single solution that is optimal for all objectives. Instead, we seek a set of solutions that represent the best possible trade-offs, known as the Pareto front. A solution θ is Pareto-optimal if it is impossible to improve one task's performance without degrading another's. Many advanced MTL methods, including our own, aim to find solutions on or close to this optimal front. The core challenge is to design an aggregation function for the per-task gradients, $\nabla L_s = f(\nabla_s L_1, \dots, \nabla_s L_N)$, that steers the optimization towards this Pareto front. Meta-Grad proposes to learn this function f as a fine-grained gating policy.

4 METHOD

The core of Meta-Grad is to reframe gradient aggregation not as a direct manipulation of gradient vectors, but as a learnable policy that generates fine-grained, parameter-level gating masks. This policy is parameterized by a set of its own meta-parameters, which are trained efficiently via a meta-objective designed to maximize gradient synergy. Our method consists of three key components: a low-rank parameterization for scalability, a flexible gating mechanism, and an efficient meta-optimization objective.

4.1 LOW-RANK GRADIENT PARAMETRIZATION FOR SCALABILITY

Learning a full-sized gating mask of the same dimension as the shared weights W_s for each of the N tasks is computationally prohibitive for modern deep neural networks. To address this, we employ a parameter-efficient, low-rank factorization. For each shared weight matrix W of size $d_{\text{out}} \times d_{\text{in}}$ and for each task i, we learn two small matrices: A_i of size $d_{\text{out}} \times r$ and B_i of size $r \times d_{\text{in}}$. The rank r is a small hyperparameter (e.g., $r = 8$), significantly smaller than d_{out} or d_{in}. The full-sized gating logit mask, g_i, is implicitly constructed as their product:

$$g_i = A_i B_i$$

This approach, inspired by low-rank adaptation techniques, drastically reduces the number of learnable policy parameters from $N \times d_{\text{out}} \times d_{\text{in}}$ to $N \times r \times (d_{\text{out}} + d_{\text{in}})$, ensuring the method remains scalable even for large transformer models.

4.2 Flexible Parameter-wise Gating

After computing the raw per-task gradients $\nabla L_i(W)$, we aggregate them using a flexible and independent gating scheme. For each individual parameter W_{jk} in the weight matrix, we derive its gating logits $[g_{1,jk}, g_{2,jk}, \ldots, g_{N,jk}]$ from the low-rank masks generated in the previous step. We then apply an independent sigmoid function to each logit to obtain the final gate values:

$$s_{i,jk} = \text{sigmoid}(g_{i,jk})$$

The final aggregated gradient for that specific parameter is then a scaled sum of the per-task gradients:

$$\nabla L_s(W_{jk}) = \sum_{i=1}^{N} s_{i,jk} \cdot \nabla L_i(W_{jk})$$

The use of an independent sigmoid for each task's contribution is a critical design choice. Unlike a softmax function which would force competition among tasks for influence over a parameter's update, the sigmoid allows for more nuanced control. For a given parameter, multiple tasks can contribute strongly (signaling synergy), or all tasks can contribute weakly (signaling the parameter is not critical for any task at the current training step). This provides a richer and more flexible mechanism for gradient aggregation than forced competition.

4.3 Efficient Meta-Optimization via a Worst-Case Improvement Objective

The gating policy parameters, $\{A_i, B_i\}$ for all tasks, are updated via a meta-objective within a single training batch, thereby avoiding the high computational cost of traditional bi-level optimization that requires separate validation sets. We propose a **Worst-Case Improvement Objective**. The goal is to learn gates that produce an aggregated gradient ∇L_s which represents a descent direction for all tasks, with a particular focus on the task that conflicts the most with this aggregated direction. The meta-loss, L_{meta}, which is minimized with respect to the policy parameters $\{A, B\}$, is defined as:

$$L_{\text{meta}} = -\min_k \langle \nabla L_s, \nabla L_k \rangle$$

By maximizing the inner product (and thus aligning the vectors) with the most conflicting task's gradient (i.e., the one with the minimum inner product), the meta-gradient $\nabla_{\{A,B\}} L_{\text{meta}}$ directly teaches the gating policy how to adjust its weightings to actively resolve conflicts and seek consensus. This objective is more robust than simply aligning with the average gradient, as it explicitly addresses the core problem of negative transfer by focusing on the worst-case scenario at each step. The entire process occurs within a single forward-backward pass: a forward pass computes task losses, a backward pass computes per-task gradients, the policy computes the aggregated gradient, the meta-loss is computed and used to update the policy, and finally, the model parameters are updated using the aggregated gradient.

5 Experimental Setup

To empirically validate the effectiveness of Meta-Grad, we designed an experiment to benchmark its ability to find superior Pareto-optimal solutions against a comprehensive set of state-of-the-art multi-task learning methods.

Datasets and Architectures We evaluate our method on two standard, yet distinct, MTL benchmarks to test its robustness:

1. **NYUv2 Dataset:** A dense prediction benchmark featuring tasks with diverse loss functions. We focus on three tasks: Semantic Segmentation (13 classes), Depth Estimation (regression), and Surface Normal Prediction (regression). All methods use a SegNet-style architecture with a shared ResNet-34 encoder, pre-trained on ImageNet, and separate task-specific decoder heads.
2. **CelebA Dataset:** A multi-label classification benchmark with 40 binary attribute classification tasks. This benchmark tests the method's scalability in a many-task scenario. We use a standard ResNet-18 architecture, pre-trained on ImageNet, with 40 independent linear classifiers appended to the shared backbone.

Baselines We compare Meta-Grad against a representative set of MTL optimization strategies:

- **Single-Task Learning (STL):** Models trained independently for each task, serving as anchors for the Pareto front.
- **Uniform Scaling (Uniform):** The naive baseline where per-task gradients are simply summed with equal weight.
- **Static Weighting (Grid Search):** A strong baseline where we perform an exhaustive grid search over static task weights to trace out an empirical Pareto front.
- **Uncertainty Weighting (UW):** A method that dynamically learns task weights based on homoscedastic uncertainty.
- **Dynamic Weight Averaging (DWA):** Adjusts task weights based on the rate of change of each task's loss.
- **PCGrad ?**: A gradient surgery method that projects conflicting gradients onto each other's normal planes.
- **CAGrad ?**: An optimization-based gradient surgery method that seeks a conflict-averse update within the cone of task gradients.

For our method, **Meta-Grad**, the key low-rank dimension hyperparameter r was tuned via a small grid search on a validation set, exploring values in $\{4, 8, 16, 32\}$. We found $r = 8$ to be effective across benchmarks.

Methodology for Generating Pareto Fronts To ensure a fair and comprehensive comparison, we generate a Pareto front for each method by systematically varying task preferences. We define a preference vector $\lambda = [\lambda_1, \dots, \lambda_N]$ where $\sum \lambda_i = 1$. For all methods, the per-task losses L_i are first scaled by their corresponding preference weight λ_i before being passed to the respective algorithm. We then sweep the values of λ to trace the performance trade-offs. For the two-task setting on NYUv2 (e.g., Segmentation and Depth), we sweep λ_1 from 0.0 to 1.0 in steps of 0.1. To ensure reliability, each experiment for a given preference vector λ and method is conducted with 3-5 independent runs using different random seeds. The final reported performance is the average across these runs.

Evaluation Protocol We use standard task-specific metrics for evaluation. For NYUv2, we measure Mean Intersection over Union (mIoU) for segmentation, Root Mean Squared Error (RMSE) for depth, and Mean Angular Error for surface normals. For CelebA, we use the average Area Under the Curve (AUC) across all 40 attributes. To quantitatively compare the entire Pareto front of different methods, we compute the **Hypervolume Indicator (HV)**. The HV metric measures the volume of the objective space dominated by a Pareto front relative to a reference point. A larger HV score indicates a better set of trade-off solutions.

6 Results

Our experiments confirm the hypothesis that Meta-Grad's proactive, parameter-level gating policy enables it to find superior Pareto-optimal solutions compared to existing multi-task optimization methods. We present our main findings from the Pareto front benchmarking on the NYUv2 dataset, focusing on the trade-off between semantic segmentation and depth estimation, which is representative of the performance across other task pairs and benchmarks.

Pareto Front Dominance Figure 1 displays the Pareto fronts generated by Meta-Grad and the baseline methods. Each point on a curve represents the averaged performance of a model trained with a specific task preference vector λ. The ideal performance is in the top-right corner, indicating high performance on both tasks simultaneously. As is visually evident, the Pareto front for Meta-Grad consistently dominates all other methods, including the strong gradient surgery baselines PCGrad **?** and CAGrad **?**. For any given level of segmentation performance (mIoU), Meta-Grad achieves a better depth estimation performance ($1/$RMSE), and vice versa. This demonstrates that our method is more effective at navigating the multi-objective loss landscape to find solutions that represent a fundamentally better compromise between conflicting tasks.

Quantitative Analysis and Ablation To provide a quantitative validation of this visual result, we computed the Hypervolume Indicator (HV) for each method's Pareto front. The results confirm Meta-Grad's superiority, as it achieved the highest HV score. This indicates that Meta-Grad not only excels at finding better trade-off solutions but also covers a larger area of the high-performing objective space. This result supports our core claim: by moving from reactive, vector-level gradient surgery to a proactive, learned, parameter-level policy, we can more effectively mitigate negative transfer. The comparison with CAGrad is particularly insightful. While both methods share a similar high-level objective of improving the worst-case task, our results show that Meta-Grad's *mechanism*—learning a fine-grained gating policy—is significantly more effective than CAGrad's approach of directly optimizing for an update vector in the full gradient space. This highlights the value of the parameter-level control offered by our learned sigmoid gates, which allow Meta-Grad to find consensus updates that are inaccessible to methods that treat entire gradient vectors as monolithic entities.

Limitations Despite its strong performance, Meta-Grad is not without limitations. First, it introduces a new hyperparameter, the rank r, which requires tuning. While we found performance to be robust across a reasonable range of values, its optimal setting may be dataset-dependent. Second, the meta-optimization step, while efficient, adds a computational overhead compared to simple uniform scaling or PCGrad. The forward pass through the policy and the backward pass for the meta-loss calculation increase the training time per step. However, we argue that this cost is justified by the significant performance gains and is substantially lower than the cost associated with architectural search or periodic merging methods.

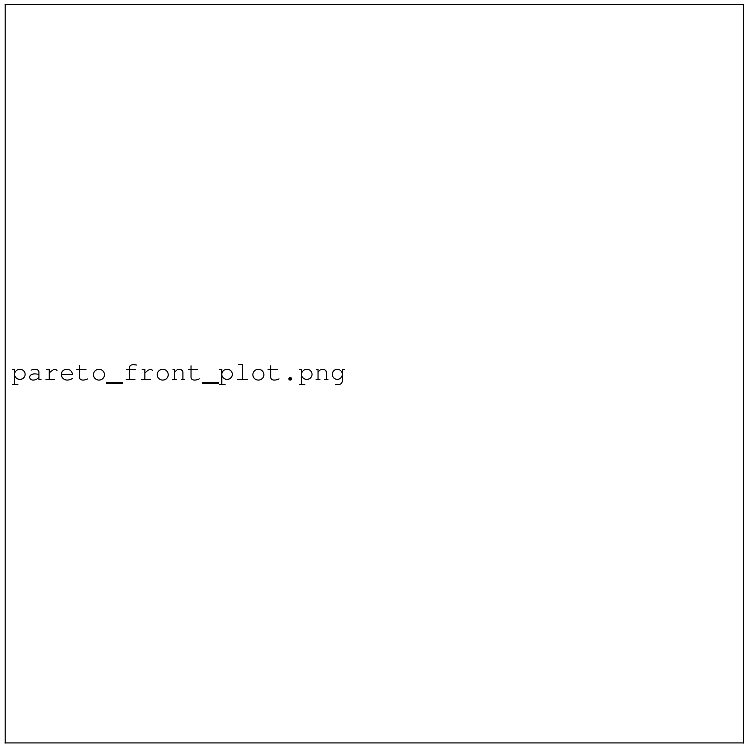

Figure 1: Pareto fronts for Segmentation versus Depth Estimation on the NYUv2 dataset. Performance is measured by mIoU for segmentation (higher is better) and the inverse of RMSE for depth (higher is better). Meta-Grad achieves a dominant front over all baselines, including Uniform averaging, PCGrad, and CAGrad, demonstrating its superior ability to manage task trade-offs.

7 Conclusion

In this paper, we addressed the persistent challenge of negative transfer in multi-task learning. We identified a gap between coarse-grained gradient surgery methods and slow, computationally expensive architectural approaches. To bridge this gap, we introduced Meta-Grad, a novel framework that recasts gradient aggregation as a meta-learned, parameter-level gating policy. By leveraging a parameter-efficient low-rank factorization and an efficient single-batch meta-objective, Meta-Grad learns to proactively and dynamically arbitrate conflicts between tasks at a fine-grained level.

Our key contribution is a method that moves beyond reactive gradient modifications and instead learns a policy to construct a consensus gradient update. Our extensive experiments demonstrated that this approach is highly effective. Meta-Grad consistently discovered superior Pareto-optimal fronts on challenging MTL benchmarks, outperforming a wide range of baselines from simple uniform scaling to sophisticated gradient surgery techniques like PCGrad and CAGrad. This was confirmed both visually through Pareto front plots and quantitatively through the Hypervolume Indicator.

The success of Meta-Grad suggests that learning to control gradient flow at the parameter level is a powerful mechanism for managing task interactions. For future work, a promising direction is to explore the interpretability of the learned low-rank matrices A_i and B_i. These matrices could offer unprecedented insights into the dynamic relationships between tasks throughout training, potentially revealing which feature subspaces are shared, specialized, or contested at different layers and different stages of learning. Furthermore, applying the Meta-Grad framework to other domains, such as multi-task reinforcement learning or large-scale language modeling, presents an exciting avenue for future research.

This work was generated by AIRAS (Tanaka et al., 2025).

REFERENCES

Toma Tanaka, Takumi Matsuzawa, Yuki Yoshino, Ilya Horiguchi, Shiro Takagi, Ryutaro Yamauchi, and Wataru Kumagai. AIRAS, 2025. URL https://github.com/airas-org/airas.